

Day 3, Session 1: Distributed Computing with MPI

(`mpi4py`)

Duration: 2 hours (theory + live demonstrations)

Prerequisites: Days 1 & 2 (Processes, Serialization, Inter-process Communication)

Demos folder: `day3/demo/`

Table of Contents

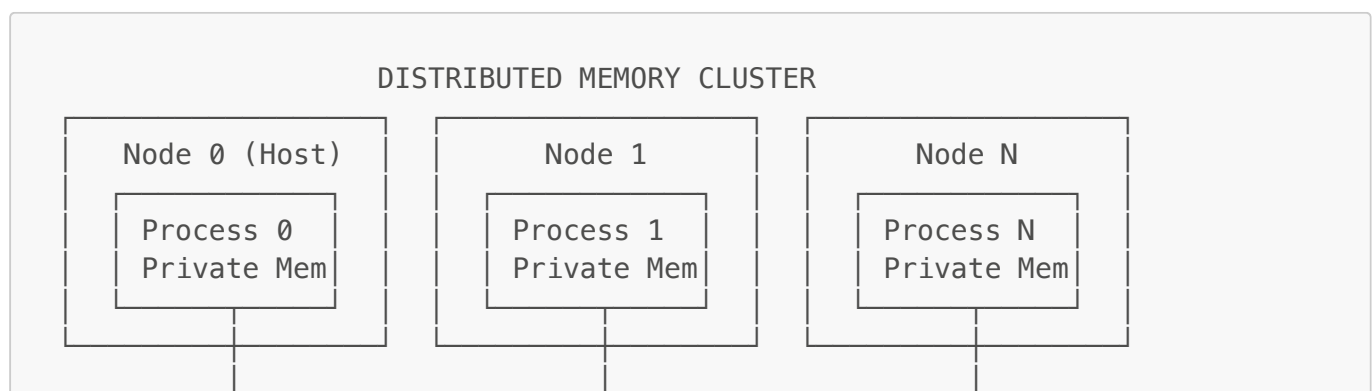
1. Understanding MPI & Distributed Memory
2. Setting up and Running MPI in Python (`mpi4py`)
3. Communicators, Ranks, and Size
4. Point-to-Point Communication
5. Blocking vs Non-Blocking Communication
6. Avoiding Deadlock Problems
7. Collective Communication Operations
 - Broadcast (`bcast`)
 - Scatter (`scatter`)
 - Gather (`gather`)
 - Reduction (`reduce` & `allreduce`)
 - All-to-All (`alltoall`)
8. Optimizing MPI Communication
9. Session 1 Summary

1. Understanding MPI & Distributed Memory

Up to this point, our parallel programs ran on a **single machine** (shared-memory systems):

- In Day 1 & 2, multiple threads shared one address space.
- Multiple processes each had their own address space on the same machine, using OS pipes/queues.

In High-Performance Computing (HPC), problem sizes often exceed what a single machine can store or compute. We connect dozens, hundreds, or thousands of computers (nodes) over high-speed networks (InfiniBand, Ethernet) into a **cluster**.



High-Speed Interconnect / Network (MPI)

What is MPI?

MPI stands for **Message Passing Interface**. It is a standardized, portable message-passing system designed by a consortium of academic and industrial researchers:

- It is the de-facto standard for parallel programming on supercomputers and clusters.
- Programs run as multiple instances of the same code (**SPMD**: Single Program, Multiple Data).
- Each process has an ID called its **Rank**.
- Communication is explicit: processes exchange messages across the network.

2. Setting up and Running MPI in Python (**mpi4py**)

Python interfaces with MPI through the standard package **mpi4py**:

```
# On macOS (via Homebrew + pip):  
brew install open-mpi  
pip3 install mpi4py  
  
# On Ubuntu/Debian:  
sudo apt install libopenmpi-dev openmpi-bin  
pip3 install mpi4py
```

Running MPI Programs

Unlike standard Python scripts that you execute with **python3 script.py**, MPI programs are launched by an MPI runner, typically **mpirun** or **mpiexec**:

```
mpirun -n 4 python3 my_script.py
```

This command starts **4 independent operating system processes**, all executing **my_script.py** simultaneously!

3. Communicators, Ranks, and Size

When **mpi4py** initializes, it groups all launched processes into an environment called a **Communicator**.

- **Communicator** (**MPI.COMM_WORLD**): The group of all active processes.
- **Rank** (**comm.Get_rank()**): The unique integer ID of the current process (from **0** to **size - 1**).
- **Size** (**comm.Get_size()**): The total number of processes launched in the communicator.

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

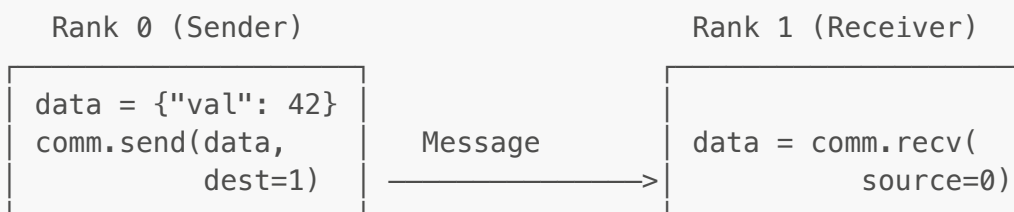
print(f"Hello from Process Rank {rank} of {size} total processes!")
```

```
Launch: mpirun -n 3 python3 script.py
Output:
Hello from Process Rank 0 of 3 total processes!
Hello from Process Rank 1 of 3 total processes!
Hello from Process Rank 2 of 3 total processes!
```

4. Point-to-Point Communication

Point-to-point communication involves **exactly two processes**:

1. A **Sender** that targets a specific destination rank.
2. A **Receiver** that accepts messages from a specific source rank.



In `mpi4py`, lowercase methods (`send`, `recv`) automatically use **Pickle** under the hood, allowing you to send arbitrary Python objects:

```
# Sender (Rank 0)
if rank == 0:
    data = {"task": "matrix_multiply", "params": [100, 200]}
    comm.send(data, dest=1, tag=10)

# Receiver (Rank 1)
elif rank == 1:
    received = comm.recv(source=0, tag=10)
    print("Received:", received)
```

Tags: An optional integer tag distinguishes between different kinds of messages sent between the same two processes.

5. Blocking vs Non-Blocking Communication

Blocking (`comm.send` / `comm.recv`)

A **blocking** call does not return until the resources are safe to reuse:

- `comm.send()` waits until the message has either been copied into an MPI system buffer or delivered to the receiver.
- `comm.recv()` blocks execution until the message has arrived and been unpacked.

Non-Blocking (`comm.isend` / `comm.irecv`)

Non-blocking calls initiate the transfer and return **immediately**, yielding a **Request** handle. This allows the CPU to compute while data transfers over the network (**overlapping computation and communication**).

Non-Blocking Overlap:

Process 0:

```
|— comm.isend(...) —> returns Request immediately  
|— [Heavy CPU Math / Local Processing] <— (Transfers concurrently!)  
|— req.wait()      —> ensures data is transmitted before moving on
```

```
# Non-blocking Send  
req = comm.isend(large_data, dest=1, tag=20)  
# Do other work here while data is being transmitted!  
do_useful_work()  
req.wait() # Block until the send is finished
```

6. Avoiding Deadlock Problems

A **deadlock** occurs when two or more processes are mutually blocked waiting for each other to take action.

The Classic Deadlock Scenario

Imagine Rank 0 and Rank 1 both want to swap data:

```
# BAD CODE — Potential Deadlock!  
if rank == 0:  
    comm.send(data_0, dest=1) # May block waiting for buffer/receiver  
    comm.recv(source=1)  
elif rank == 1:  
    comm.send(data_1, dest=0) # May block waiting for buffer/receiver  
    comm.recv(source=0)
```

If the messages are large and exceed MPI's internal buffer, both processes will block on `send()` waiting for the other process to call `recv()`. Neither ever reaches `recv()`, causing the program to freeze forever!

Solutions

1. Alternate Send / Recv Order:

- Rank 0: `send()` then `recv()`
- Rank 1: `recv()` then `send()`

2. Use Non-Blocking Calls (`isend` / `irecv`):

Start both operations without blocking.

3. Use `comm.sendrecv()`:

A specialized atomic MPI call that performs a simultaneous send and receive without deadlocking:

```
received = comm.sendrecv(sendobj=my_data, dest=peer, source=peer)
```

Demo Time

Now, let's see these theoretical topics in action.

- `01_mpi_point_to_point.py` — Run with: `mpirun -n 2 python3 01_mpi_point_to_point.py`
 - `demo_basic_send_recv()` — Simple message passing between two ranks
 - `demo_nonblocking_isend()` — Overlapping computation with `isend` / `irecv`
 - `demo_deadlock_avoidance()` — Safe bilateral data swapping with `comm.sendrecv()`

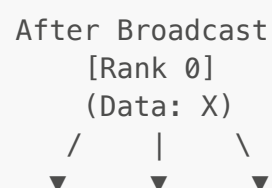
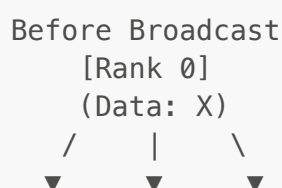
7. Collective Communication Operations

While point-to-point sends data between two specific ranks, **collective operations** involve **all processes** within a communicator simultaneously.

Crucial Rule: All processes in the communicator **MUST** participate and call the collective method. If even one rank skips it, the other ranks will hang waiting!

Broadcast (One-to-All)

The root process transmits identical data to every process in the communicator.



[Rank 1] [Rank 2] [Rank 3]
(empty) (empty) (empty)

[Rank 1] [Rank 2] [Rank 3]
(Data: X) (Data: X) (Data: X)

```
if rank == 0:
    config = {"threshold": 0.85, "learning_rate": 0.001}
else:
    config = None

config = comm.bcast(config, root=0)
```

Scatter (Divide and Distribute)

The root process takes an iterable (array/list), splits it into equal parts, and gives one chunk to each rank.

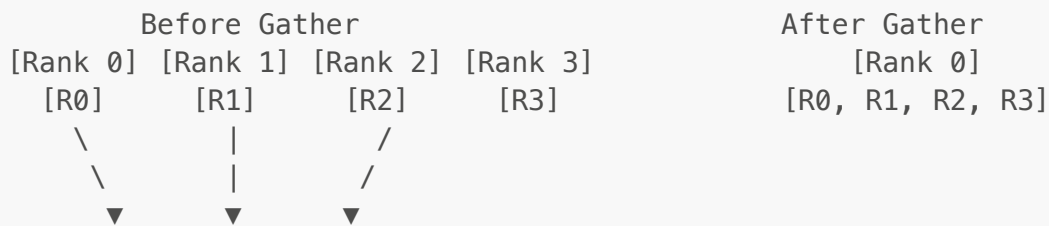


```
if rank == 0:
    data = [10, 20, 30, 40] # list length must equal size!
else:
    data = None

my_chunk = comm.scatter(data, root=0)
```

Gather (Collect and Combine)

The inverse of scatter. Every process holds a piece of data, and the root process collects all pieces into a single ordered list.



```
# Each rank computes its own answer
my_answer = rank * 100

# Gather to root
all_answers = comm.gather(my_answer, root=0)
# Rank 0 gets: [0, 100, 200, 300]
```

Reduction (Aggregate Values)

All processes contribute a value, and MPI applies an associative mathematical operator (such as **SUM**, **MAX**, **MIN**, **PROD**) across all values.

- `comm.reduce(..., root=0)`: Only the root rank receives the aggregated final number.
- `comm.allreduce(...)`: **All** ranks receive the final aggregated result.

```
Rank 0: 5
Rank 1: 12
Rank 2: 8
Rank 3: 6
```

→ MPI.SUM → Total: 31 (stored on Rank 0 or all ranks)

```
local_loss = compute_loss()
total_loss = comm.reduce(local_loss, op=MPI.SUM, root=0)
global_max = comm.allreduce(local_loss, op=MPI.MAX) # available on every rank!
```

All-to-All (Personalized Exchange)

Every rank has a distinct message for every other rank. It is essentially an all-process matrix transpose.

```
# Each rank prepares a list of messages of length 'size'
send_data = [f"From {rank} to {i}" for i in range(size)]
received = comm.alltoall(send_data)
```

8. Optimizing MPI Communication

In Python, `mpi4py` offers two distinct API styles:

Feature	Lowercase (<code>send</code> , <code>recv</code> , <code>bcast</code>)	Uppercase (<code>Send</code> , <code>Recv</code> , <code>Bcast</code>)
Data types	Arbitrary Python objects	Contiguous memory buffers (NumPy)
Mechanism	Pickles to byte string	Direct zero-copy C memory transfer
Overhead	High (serialization + alloc)	Near zero (C-level speeds)
Syntax	<code>comm.send(obj, dest)</code>	<code>comm.Send([np_array, MPI.DOUBLE], dest)</code>

HPC Best Practice: When processing large arrays or matrices in HPC applications, always use NumPy arrays with uppercase methods (`comm.Send`, `comm.Recv`, `comm.Scatter`, `comm.Gather`). This achieves raw C / Fortran network transfer speeds!

Demo Time

Now, let's see these theoretical topics in action.

- `02_mpi_collective.py` — Run with: `mpirun -n 4 python3 02_mpi_collective.py`
 1. `demo_broadcast()` — Distribute configuration from Rank 0 to all ranks
 2. `demo_scatter_gather()` — Distribute work, compute locally, and gather results
 3. `demo_reduction()` — Sum reduction and global max with `reduce` / `allreduce`
 4. `demo_alltoall()` — Personalized all-to-all communication

Session 1 Summary

Operation	Method	Description
Send / Receive	<code>comm.send</code> , <code>comm.recv</code>	Point-to-point transfer between 2 ranks
Non-blocking	<code>comm.isend</code> , <code>comm.irecv</code>	Initiates transfer; allows compute overlap
Send-Receive	<code>comm.sendrecv</code>	Atomic two-way swap; avoids deadlocks
Broadcast	<code>comm.bcast</code>	Sends 1 piece of data to all ranks
Scatter	<code>comm.scatter</code>	Splits a list evenly across all ranks
Gather	<code>comm.gather</code>	Assembles chunks from all ranks into 1 list
Reduction	<code>comm.reduce</code> , <code>comm.allreduce</code>	Computes aggregate (<code>SUM</code> , <code>MAX</code> , etc.)
All-to-all	<code>comm.alltoall</code>	Personalized exchange (matrix transpose)

Break time! Next up is Session 2: Asynchronous Programming (`concurrent.futures` & `asyncio`) → [README-session2.md](#)